

I/O 系统：请求、搬运、等待与完成

从系统调用、轮询、中断、DMA 到缓冲、页缓存、假脱机与磁盘调度

408 专题手册 · 操作系统 I/O 篇

核心模型：异步设备怎样接入指令执行过程

CPU 与进程按指令流推进，而设备速度各异、独立工作、完成时间不确定。

I/O 系统的任务，是把“请求、等待、搬运、完成、缓存、排队”组织成一个可控的状态机。

分析主线：

```
Request → Submit → Wait / Continue → Transfer
          → Complete → Wake / Notify → Return
```

做任何 I/O 题，都同时追踪三条线：

Control Flow

Data Flow

Process State Flow

目录

1 第 0 章：为什么 I/O 必须成为独立子系统	2
1.1 五个根本矛盾	2
1.2 三条并行线：做综合题时必须同时看	2
2 第 1 章：I/O 请求路径与完成路径	3
2.1 请求怎样进入内核	3
2.2 完成怎样重新进入内核	3
2.3 中断、同步异常与“Trap”术语边界	3
2.4 Mode Switch 不等于 Context Switch	4
3 第 2 章：从最原始的设备交互开始	4
3.1 控制器是 CPU 与物理设备之间的硬件接口	4
3.2 程序查询方式：最简单，也最浪费 CPU	4
4 第 3 章：中断驱动 I/O——把“等待”从 CPU 上卸载	5
4.1 为什么中断自然产生	5
4.2 一次中断驱动 I/O 的完整状态路径	5

4.3 中断本身也有成本	5
5 第 4 章: DMA 与 I/O 通道——把“搬运”进一步卸载	6
5.1 Interrupt 只解决“等”，没有自动解决“搬”	6
5.2 DMA 四阶段模型	6
5.3 DMA 仍然消耗系统资源	6
5.4 408 经典: DMA 的几种总线使用方式	7
5.5 I/O Channel: 比 DMA 更进一步的控制卸载	7
5.6 四种经典 I/O 控制方式总对比	8
5.7 经典 I/O 通道类型: 按“设备共享通道的方式”理解	8
6 第 5 章: Driver 与 Device-independent I/O——把设备知识隔离	8
6.1 为什么必须有驱动	8
6.2 经典 I/O 软件层次	9
6.3 字符设备与块设备: 不同工作负载产生不同接口	9
6.4 I/O 保护: 为什么设备寄存器和 DMA 也需要权限边界	9
6.5 错误与完成码: I/O 慢路径不只有“成功”	10
7 第 6 章: 四个维度必须拆开——Blocking、Async、Interrupt、DMA	10
7.1 为什么这几个词特别容易乱	10
7.2 阻塞与非阻塞只看“当前线程”	10
7.3 同步与异步要看“完成语义”	11
8 第 7 章: Buffer、Cache、Spooling——看起来都“存一下”，目的完全不同	11
8.1 Buffer: 速率匹配与粒度匹配	11
8.2 单缓冲、双缓冲与循环缓冲	12
9 第 8 章: 页缓存——文件 I/O 的内存快路径	12
9.1 核心对象: 缓存的是“文件内容”	12
9.2 读取路径: Hit、Miss 与 Read-ahead	12
9.3 写入路径: Buffered Write 与 Dirty Page	13
9.4 三个时间点必须分开	13
9.5 fsync() 不是简单的 Write-through	13
9.6 回收页缓存: 问“丢掉以后怎样恢复”	13
10 第 9 章: Spooling——把独占慢设备虚拟化成可排队服务	14
10.1 更深的定义: 不是“磁盘缓存打印机”	14
10.2 经典输出 Spooling 架构模型	14

10.3 408 经典输入/输出井模型	14
11 第 10 章：设备分配与 I/O 调度——多个请求如何分享有限设备	15
11.1 独占、共享与虚拟设备	15
11.2 设备分配涉及的资源对象	15
11.3 408 经典设备分配数据结构：把“设备链”画出来	15
11.4 静态分配与动态分配	15
11.5 为什么块设备需要调度	15
12 第 11 章：HDD 成本模型与经典磁盘调度	16
12.1 先有物理成本，再有调度策略	16
12.2 六种经典算法的“设计动机”	16
12.3 磁盘组织与管理：调度之外还要知道哪些系统动作	17
12.4 SSD 之后为什么经典电梯思想变弱	17
13 第 12 章：综合题——一次阻塞 read() 的完整路径	17
13.1 场景设定	17
13.2 完整流程架构图	18
13.3 三条时间线合并	18
14 第 13 章：高频概念辨析与错因诊断	18
15 第 14 章：408 I/O 解题检查表	20
15.1 题型到核心模型的映射	20
16 第 15 章：知识树与查漏清单	21
16.1 408 Core：必须形成自动反应	21
16.2 Mechanism Layer：理解后做综合题更稳	21
16.3 工程扩展：用于限定教材模型的边界	22
17 第 16 章：专题接口	22
17.1 与“进程、线程、调度与控制权”专题	22
17.2 与“CPU × OS 软硬件协作”专题	22
17.3 与“内存虚拟化”专题	22
17.4 与下一本“文件系统”专题	22
18 核心结论：I/O 是异步状态转换系统	23
参考与工程边界资料	24

1 第 0 章：为什么 I/O 必须成为独立子系统

1.1 五个根本矛盾

I/O 不是“CPU 外面挂几个设备”这么简单。它之所以形成完整子系统，是因为至少同时存在五种矛盾。

根本矛盾	如果不解决会怎样	自然产生的机制 / 数据结构
速度不匹配	CPU 被慢设备拖住；小粒度请求频繁穿越慢路径	Buffer、Cache、Queue、Read-ahead
完成时间不确定	CPU 不知道设备什么时候完成，只能反复询问	Interrupt、Completion、Wait/Wakeup
搬运成本高	CPU 花大量指令在设备与内存之间重复搬数据	DMA (Direct Memory Access)、I/O Channel
设备接口异构	上层必须理解每种控制器寄存器和协议	Driver 驱动程序、Device-independent I/O
资源有限/独占	多个进程争一个打印机、磁盘队列或设备通道	Scheduling 磁盘调度、Spooling、Device Allocation

I/O 的四次“卸载”

把 I/O 技术按“CPU/进程被什么拖住”来理解，会比按年代背诵更稳定：

Polling → Interrupt :	卸载等待
PIO → DMA :	卸载搬运
Device-specific → Driver :	卸载设备知识
Direct coupling → Buffer/Cache/Spool :	卸载时间与速率耦合

1.2 三条并行线：做综合题时必须同时看

控制线 Control Flow 谁当前拥有 CPU？控制权怎样进入内核，又怎样从设备完成事件回到内核？

User → Syscall → Kernel Device → Interrupt → Kernel

数据线 Data Flow 数据到底从哪里移动到哪里？谁执行搬运？

Device $\xrightarrow{\text{DMA}}$ RAM/页缓存 $\xrightarrow{\text{copy}}$ User Buffer

状态线 Process State Flow 请求发出以后，原执行流继续、忙等、阻塞还是稍后被唤醒？

Running → Blocked → Ready → Running

三条线最常见的混淆

“DMA 完成”描述数据搬运；“设备发中断”描述完成通知；“ISR 把等待进程唤醒”描述进程状态变化。三者通常前后相连，但绝不是同一件事。**I/O 完成也不等于原进程立即 Running**：通常只是 Blocked → Ready，是否马上获得 CPU 仍由调度器决定。

2 第 1 章：I/O 请求路径与完成路径

2.1 请求怎样进入内核

用户进程不能直接随意编程设备控制器、修改中断状态或操作关键特权资源。因此 I/O 请求通常通过受控系统调用入口进入内核：

User Request → System Call → Kernel I/O Path

例如 `read(fd, buf, n)` 的语义入口属于用户主动发起的同步控制转移；至于系统调用具体使用 `int`、`syscall`、`svc`、`ecall` 等指令，是 ISA/ABI 细节。

2.2 完成怎样重新进入内核

设备启动以后通常独立工作。它完成后通过中断、完成队列等机制通知 CPU/内核：

Device Completion → Interrupt/Event → Kernel Completion Path

因此 I/O 最基本的对偶是：

System Call = Request enters kernel

Interrupt = asynchronous event enters kernel

2.3 中断、同步异常与“Trap”术语边界

维度	外部中断 Interrupt	同步异常 Exception	系统调用/受控陷入
触发关系	与当前指令语义无关，外部异步到来	由当前指令执行引起	当前指令有意请求特权服务
典型来源	时钟、网卡、磁盘完成、键盘	Page Fault、除零、非法指令、保护错误	<code>syscall/svc/ecall</code> /历史 <code>int</code>
核心判断	异步事件	同步事件	同步且主动
处理后动作	返回原执行流或调度其他任务	修复后重启、向进程报告错误或终止	返回系统调用下一阶段或阻塞/调度

408 语言与体系结构严格术语要分层

教材常把“用户程序主动执行特殊指令进入内核”称为**陷入 Trap**，这在理解系统调用时完全可用；但不要把“Interrupt/Trap/Exception”当成所有 ISA 都完全一致的三分法。真正稳定的判断轴是：**同步/异步、主动/非主动、能否修复、体系结构规定的 restart point 在哪里。**

2.4 Mode Switch 不等于 Context Switch

一次系统调用或中断首先意味着：

User/Current Context $\rightarrow$ Kernel Handler

它可能仍然属于同一个进程/线程的执行上下文。只有当内核进一步决定把 CPU 交给另一个可运行实体时，才发生：

Context Switch

所以：

Interrupt/Syscall $\nrightarrow$ Context Switch

3 第 2 章：从最原始的设备交互开始

3.1 控制器是 CPU 与物理设备之间的硬件接口

典型设备控制器至少暴露三类逻辑信息：

- **Status**：忙/空闲、错误、完成状态；
- **Command/Control**：读、写、启动、复位等命令；
- **Data/Descriptor**：数据寄存器、DMA 描述符地址、队列门铃等。

CPU/驱动通过端口 I/O 或 MMIO 等方式访问这些接口。

工程边界：MMIO 比“某个固定端口”更通用

现代设备大量通过 Memory-Mapped I/O 暴露寄存器，CPU 发出的特定地址访问会被互连/总线送到设备而非普通 DRAM。408 题目更关注“控制/状态/数据寄存器 + 驱动访问”的逻辑模型，不必把某一总线实现背成唯一形式。

3.2 程序查询方式：最简单，也最浪费 CPU

最直接方案：

```
start_device();
while (status() == BUSY) {
    ; // busy waiting
}
read_result();
```

它的本质是：

CPU repeatedly pays to discover that nothing changed

如果设备操作远慢于 CPU，绝大多数检查都是无效工作。

程序查询 I/O 的核心特征

- CPU 主动反复查询设备状态；
- CPU 与设备工作可以在时间上交错，但等待期间 CPU 被忙等占住；
- 数据搬运通常仍由 CPU 指令参与；
- 实现简单，适合极短等待、简单控制器或某些低延迟特殊场景。

4 第 3 章：中断驱动 I/O——把“等待”从 CPU 上卸载

4.1 为什么中断自然产生

如果设备完成时间不可预测，最合理的协议不是 CPU 一直问，而是：

Start → Do Other Work → Device Notifies Completion

于是：

Polling → Interrupt = Offload Waiting

4.2 一次中断驱动 I/O 的完整状态路径

假设进程 A 发起阻塞式设备读取：

A : Running → syscall → driver starts I/O → Blocked

CPU 转去运行 B。设备完成以后：

Device → IRQ → ISR/Completion → A : Ready

未来调度器选中 A：

A : Ready → Running → return to user

4.3 中断本身也有成本

中断不是“免费通知”。至少可能包含：

- 保存必要体系结构状态并进入内核；
- 执行 ISR/完成处理；
- cache/TLB/流水线局部扰动；
- 可能产生调度和上下文切换。

因此，若事件极短、马上完成，短时间 polling/spin 可能比睡眠再唤醒更经济。

与并发专题同构: Polling vs Interrupt

两组问题本质相同:

短等待: 支付 CPU 时间, 避免切换成本

长等待: 放弃 CPU, 支付阻塞/唤醒/调度成本

所以不存在脱离等待时间与负载的“interrupt 永远优于 polling”。

现代工程: Interrupt moderation / batching

高速网卡、NVMe 等设备可能每秒完成海量小请求。若每个完成都单独中断, CPU 会被中断风暴淹没。因此现实系统常批量完成、合并中断或使用 polling 与 interrupt 的混合模式。它们改变的是工程快路径, 不改变 408 的核心模型: 完成事件必须被系统发现, 并映射成软件状态更新。

5 第 4 章: DMA 与 I/O 通道——把“搬运”进一步卸载

5.1 Interrupt 只解决“等”, 没有自动解决“搬”

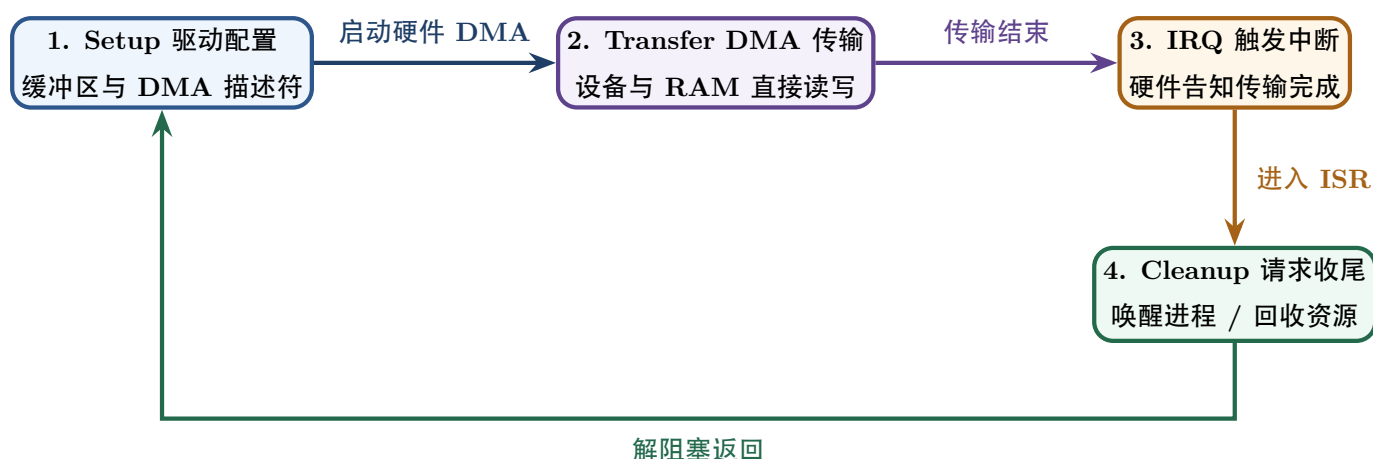
若每个数据字节仍按:

Device → CPU Register → Memory

移动, 那么 CPU 虽然不用忙等, 却还要执行大量搬运指令。因此进一步引入:

PIO → DMA = Offload Data Movement

5.2 DMA 四阶段模型



DMA 的正确一句话

DMA 不是“完全不需要 CPU”, 而是把大块数据搬运的逐字节/逐字指令参与从 CPU 上卸载。CPU 仍负责建立请求、编程/提交描述符、处理中断或完成队列, 并管理相关内存与同步状态。

5.3 DMA 仍然消耗系统资源

即使 CPU 不逐字搬数据, 传输仍然会占用:

- 内存带宽;
- 系统互连/总线带宽;
- 设备队列与控制器资源;
- 某些平台上的 cache coherence / IOMMU 映射资源。

所以:

DMA $\neq$ zero cost

工程边界: DMA address 不一定等于 CPU physical address

现代平台常存在 IOMMU 或总线地址翻译。驱动通常通过 DMA API 得到设备可使用的 DMA address; 设备使用该地址访问内存, 必要时再由 IOMMU 翻译到真实系统内存。不要把“DMA 直接写物理地址”当成所有平台的绝对实现。

5.4 408 经典: DMA 的几种总线使用方式

不同教材术语略有差异, 但核心权衡是“DMA 连续占用互连多长时间”。

- **Cycle Stealing (周期挪用)**: DMA 每次取得一个/少量总线周期, 与 CPU 交替使用;
- **Burst/Block Transfer (突发/块传送)**: DMA 一次连续传送较大数据块, 吞吐高但 CPU 可能更久拿不到总线;
- **Transparent DMA (透明方式)**: 尽量利用 CPU 不访问存储器/总线的空隙传输, 干扰小但实现和可用时机受限。

做题时不要只背名称, 要问: 谁占用总线? 持续多久? CPU 被延迟多少? 总体吞吐如何?

5.5 I/O Channel: 比 DMA 更进一步的控制卸载

在经典大型机/教材模型中, I/O 通道可以执行专门的通道程序, 管理一串 I/O 操作:

CPU issues high-level I/O job $\rightarrow$ Channel executes I/O program

相较 DMA 主要卸载数据搬运, 通道还卸载更多**设备控制与 I/O 操作序列组织**。408 若出现“通道”概念, 应抓住这个层级区别, 而不是把它简单等同于 DMA 控制器。

5.6 四种经典 I/O 控制方式总对比

方式	CPU 是否忙等	数据主要由谁搬	完成/控制粒度	核心评价
程序查询	是	CPU / PIO	CPU 反复检查, 常按小单位推进	硬件简单, CPU 浪费最大
中断驱动	否 (可去做别的)	经典模型中 CPU 仍较多参与	设备准备/完成时中断 CPU	卸载等待, 但中断和搬运仍有开销
DMA	否	DMA 引擎/设备 ↔ 内存	CPU 配置块传输, 完成后再通知	大幅减少 CPU 搬运参与
I/O 通道	否	通道执行 I/O 程序并组织设备操作	CPU 提交更高层 I/O 任务	进一步卸载控制与序列组织

最稳的比较方法：不要背“谁最好”

对每种控制方式统一问四件事：**CPU 是否忙等？CPU 是否逐单位搬数据？一次需要多少次 CPU/中断参与？额外硬件复杂度有多大？** 技术演进的主方向是降低 CPU 对慢 I/O 的细粒度参与，但硬件和协议复杂度随之上升。

5.7 经典 I/O 通道类型：按“设备共享通道的方式”理解

部分 408/教材体系还会区分：

- **字节多路通道**：多个低速设备按字节/小单位交叉共享通道，强调多路并发；
- **数组选择通道**：一次选择一个高速设备，连续完成一批数据传输后再换设备，吞吐高但通道独占时间较长；
- **数组多路通道**：兼顾块传输与多设备交叉，在多个高速设备的块操作之间复用通道。

不要把名称孤立背诵；本质仍是：一次通道控制权给谁、持续多久、传输粒度多大、并发度如何。

6 第 5 章：Driver 与 Device-independent I/O——把设备知识隔离

6.1 为什么必须有驱动

若文件系统、网络栈、应用层都直接理解每个设备寄存器：

Upper Layer × Every Device Model

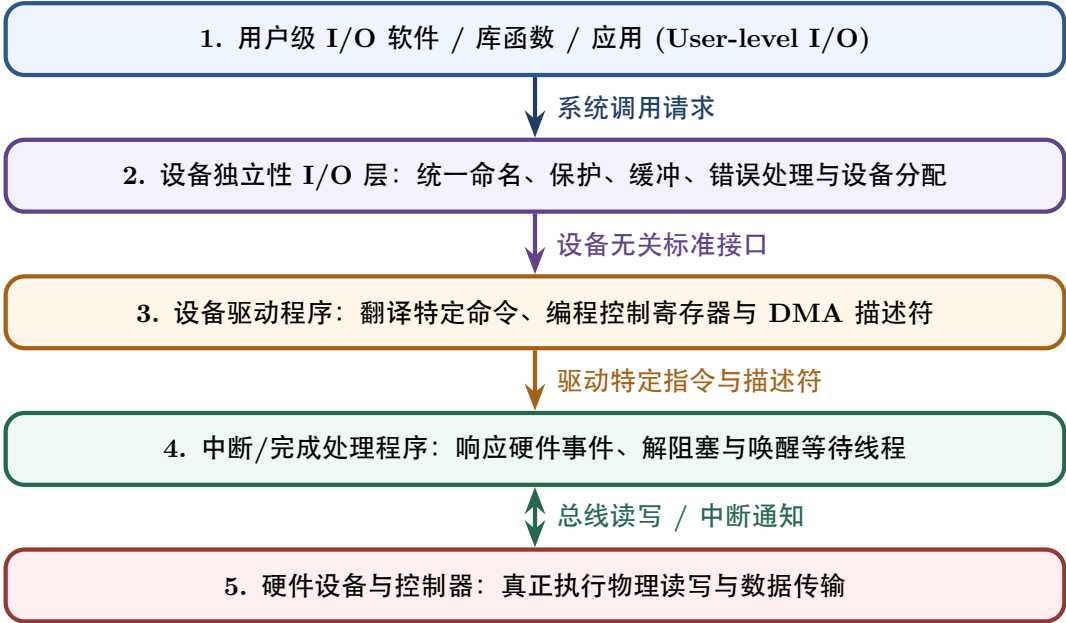
系统复杂度会爆炸。驱动建立：

Common OS Request → Device-specific Commands

于是：

Device-specific Knowledge → Driver

6.2 经典 I/O 软件层次



为什么“设备独立层”很重要

它回答的不是“某个磁盘怎么转”，而是“无论设备型号如何，上层都希望拥有统一的 open/read/write、保护、命名、缓冲、错误返回和资源分配语义”。这与 VFS、网络 socket 抽象、虚拟内存中的地址间接层是同一种复杂度控制思想。

6.3 字符设备与块设备：不同工作负载产生不同接口

维度 / 特性	字符/流式设备（经典抽象）	块设备（经典抽象）
数据组织形式	连续字节/字符流，强顺序消费与单向流动	固定大小块（扇区/扇区组），支持随机寻址
典型代表设备	键盘、串口、鼠标、部分传感器	机械硬盘 HDD、固态硬盘 SSD、块存储
OS 系统重点	流控、字符缓冲、终端行规程、事件到达	块队列、磁盘调度、页缓存缓存、块映射

实际设备/驱动接口比这更丰富，但这一区分有助于理解为什么磁盘能形成页缓存与块层，而键盘不适用“随机访问块”理解。

6.4 I/O 保护：为什么设备寄存器和 DMA 也需要权限边界

I/O 设备能够读写外部世界，某些设备还能直接 DMA 到内存，因此它们天然具有强权限。系统必须控制：

- 用户态不能任意执行特权 I/O 指令或修改关键控制器状态；
- MMIO 区域不能任意映射给不可信进程；
- DMA 缓冲和 DMA 地址必须由内核/驱动建立合法映射；
- IOMMU（若存在）可以限制设备可 DMA 的地址范围，降低设备或驱动错误破坏任意内存的风险。

因此 I/O 保护仍然符合本系列总原则:

Who can command which device to access which memory?

6.5 错误与完成码: I/O 慢路径不只有“成功”

设备完成事件可能携带: 成功、短传输、超时、校验/介质错误、设备离线等状态。内核需要决定:

Retry? Fail? Remap/Recover? Wake which waiter?

因此一个稳健的 I/O 状态机不仅有 Request → Complete, 还要允许:

Request → Pending → {Success, Retry, Error, Cancel}

408 通常不深入驱动错误恢复细节, 但做综合题时必须意识到“中断到来”只说明有事件, **不自动等于本次 I/O 成功完成**。

7 第 6 章: 四个维度必须拆开——Blocking、Async、Interrupt、DMA

7.1 为什么这几个词特别容易乱

因为它们回答的是不同问题:

维度概念	真正回答的系统核心问题	两端概念与核心对比
调用线程是否等待	调用暂时不能完成时, 进程/线程现在怎么办?	Blocking (阻塞) / Nonblocking (非阻塞)
完成语义如何交付	调用返回时结果是否已经完成? 完成以后怎样通知?	Synchronous (同步) / Asynchronous (异步)
设备完成如何被发现	CPU 主动反复查询, 还是设备主动中断通知?	Polling (程序查询) / Interrupt (中断驱动)
数据主要由谁搬运	CPU 指令逐步搬运, 还是设备/DMA 引擎搬运?	PIO (Programmed I/O) / DMA

因此完全可能有:

Blocking + Interrupt-driven + DMA

普通阻塞磁盘读取就是典型直觉模型: 线程阻塞; 设备 DMA; 完成以后中断; 最后线程被唤醒。

7.2 阻塞与非阻塞只看“当前线程”

Blocking 条件暂不满足时:

Running → Blocked

CPU 可运行别的线程。

Nonblocking 条件暂不满足时立即返回:

return EAGAIN/partial/none

应用稍后重试或结合事件机制。

7.3 同步与异步要看“完成语义”

不要把“用了中断”直接等同于“异步 I/O”。设备内部即使由中断完成，用户线程仍可能执行一个**同步阻塞**系统调用；反过来，真正异步接口通常允许请求先返回，完成以后再通过 completion queue、callback、signal/event 等方式交付结果。

四维判断法

遇到任何 I/O API/题目，不问“它到底是同步还是中断”，而依次问：

1. 调用线程等待吗？
2. 系统调用返回时数据完成了吗？
3. 设备完成事件是谁发现的？
4. 大块数据是谁搬的？

四个问题独立作答，概念自然不会打结。

8 第 7 章：Buffer、Cache、Spooling——看起来都“存一下”，目的完全不同

机制	解决的核心根本问题	核心动作与策略	典型应用场景
Buffer	生产/消费速度或粒度不匹配	暂存、吸收突发、平滑速率	键盘缓冲区、网络收发缓冲、双缓冲
Cache	慢对象可能再次被访问	保存副本以复用，命中时绕过慢层	页缓存、CPU Cache、TLB
Spooling	独占/慢设备与多个任务直接耦合	中转存储 + 排队 + 后台服务	打印任务、经典输入/输出井
DMA	CPU 逐字搬运过于昂贵	设备/控制器直接与内存传输	磁盘、网卡、GPU 数据传输
Interrupt	CPU 主动等待过于昂贵	事件发生时主动中断通知 CPU	I/O 完成、时钟、外部硬件事件

8.1 Buffer：速率匹配与粒度匹配

Buffer 的主要任务不是“下次再用”，而是让生产者和消费者不必严格同速：



它可以：

- 吸收短时突发；
- 把很多小请求聚合成大请求；
- 允许设备与 CPU 在一段时间内并行；
- 减少双方必须同时在线的程度。

8.2 单缓冲、双缓冲与循环缓冲

单缓冲 设备填充和进程取用之间仍有阶段性互斥，同一 buffer 不能一边被设备覆盖、一边被进程消费。

双缓冲

B_0 被消费时 B_1 可被设备填充

如果生产和消费时间接近，可明显提高重叠程度。

循环缓冲 多个 buffer 形成 ring:

$$B_0 \rightarrow B_1 \rightarrow \cdots \rightarrow B_{N-1} \rightarrow B_0$$

这已经自然连接到生产者-消费者问题：数据结构只是容器，正确性仍需要处理“空/满、读写索引、并发更新”。

缓冲题不要死背总时间公式

不同教材题目对“设备填充、内核复制、用户处理哪些阶段可以重叠”的假设不同。稳妥做法是：先画时间线/甘特图，再根据互斥和可重叠关系计算。公式只在假设一致时成立。

9 第 8 章：页缓存——文件 I/O 的内存快路径

9.1 核心对象：缓存的是“文件内容”

页缓存可以抽象为：

$$(File, Offset/PageIndex) \rightarrow Cached\ Memory$$

它由内核管理，使普通文件 I/O 的常见访问尽量停留在内存，而不是每次都访问底层存储。

9.2 读取路径：Hit、Miss 与 Read-ahead

Hit

read $\rightarrow$ 页缓存 Hit $\rightarrow$ copy to user $\rightarrow$ return

无底层存储读取。

Miss

页缓存 Miss $\rightarrow$ allocate/cache page $\rightarrow$ submit storage I/O

若调用是阻塞式读取，当前线程可能 Blocked。设备 DMA 把数据放入内存，完成后中断/完成处理唤醒线程，随后复制到用户 buffer。

Read-ahead 系统观察到顺序/可预测访问时，可能在应用显式请求之前预取后续文件页：

$$\text{Observed Locality} \rightarrow \text{Predict Future File Pages} \rightarrow \text{Asynchronous Prefetch}$$

不要记固定“预读 8 页/32KB”等数字；窗口和策略属于实现。

9.3 写入路径: Buffered Write 与 Dirty Page

普通 buffered write 可抽象成:

User Buffer $\rightarrow$ 页缓存 $\rightarrow$ Dirty

然后系统在稍后:

Dirty $\xrightarrow{\text{writeback}}$ Storage

这使应用不必为每次小写立即等待慢设备, 也允许多次更新合并。

9.4 三个时间点必须分开

T_{return} $T_{\text{writeback}}$ T_{durable}

- T_{return} : 系统调用对应用返回;
- $T_{\text{writeback}}$: 内核把 dirty 数据提交到底层;
- T_{durable} : 数据达到所要求的非易失持久边界。

存储设备自身还可能存在 volatile write-back cache, 因此“设备说完成”也不必然等于“已落到非易失介质”。

write() 成功返回不等于持久化完成

这是文件系统、页缓存和存储设备层次最重要的边界之一。应用若需要明确的持久性语义, 需要使用相应 durability 接口; 文件系统/块层还可能需 flush/FUA 等机制把设备易失缓存推进到非易失介质。

9.5 fsync() 不是简单的 Write-through

Write-through 是缓存策略术语, 强调每次写缓存同时推进 backing store; `fsync()` 更适合作为一个显式持久性边界请求理解: 调用方要求该文件相关的数据及必要元数据达到接口规定的持久状态。两者不能机械画等号。

9.6 回收页缓存: 问“丢掉以后怎样恢复”

- **Clean file-backed page**: 文件中已有权威副本, 通常可直接丢;
- **Dirty file-backed page**: 先写回或保留, 不能无条件丢;
- **Anonymous page**: 没有普通文件作为后备对象, 回收往往涉及 swap/压缩/其他策略。

所以不要背固定优先级, 而要问:

If evicted, how can this content be reconstructed? What is the cost?

与虚拟内存专题的接口

页缓存与 VM 在 `mmap` 处直接汇合: 文件页可以通过页表映射进进程虚拟地址空间。普通 `read()` 常见路径是页缓存 $\rightarrow$ 用户匿名 buffer 的显式复制; `file-backed mmap()` 则可以让文件页直接参与虚拟地址映射。二者最终都依赖页表、缺页和物理页生命周期。

10 第 9 章: Spooling——把独占慢设备虚拟化成可排队服务

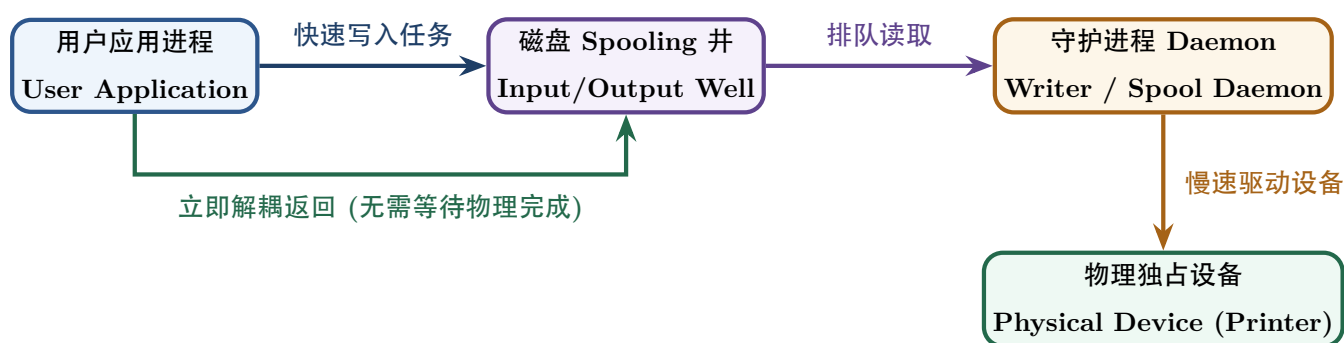
10.1 更深的定义：不是“磁盘缓存打印机”

Spooling 的核心可以写成：

Exclusive / Slow Physical Device	→	Persistent / Buffered Queue
	→	Virtual Shared Service

应用把任务提交到 spool 文件/输出队列后，不必持有物理设备直到最终完成；后台 writer/daemon 根据策略逐个把任务送到真实设备。

10.2 经典输出 Spooling 架构模型



- 时间解耦：提交任务与物理完成不要求同时；
- 资源解耦：多个任务可以先排队，不必同时争用物理设备。

10.3 408 经典输入/输出井模型

经典 SPOOLing 系统常把磁盘上的中转区称为输入井、输出井，并由输入/输出进程负责慢速外围设备与高速磁盘之间的任务搬运。考试关注的是：

- 用磁盘空间模拟独占设备的共享提交能力；
- 用户进程与慢设备解耦；
- I/O 请求进入队列，可被调度；
- 设备仍然一次服务有限任务，物理吞吐上限不会凭空提高。

Spooling 不是“打印物理速度变快”

它通常不会改变打印机每秒能打印多少页；它改变的是谁需要等待、任务何时提交、设备如何排队、CPU/用户进程能否继续做其他工作。因此它提高系统并发性和资源利用率，并显著缩短提交方占用时间，但不要把“提交完成”误当成“物理任务完成”。

思想迁移，而非概念偷换

消息队列、异步任务系统等也有“Producer → Queue → Consumer”的解耦结构，与 Spooling 在设计思想上同构。但正式术语上不应把所有队列系统都直接叫 SPOOLing。

11 第 10 章：设备分配与 I/O 调度——多个请求如何分享有限设备

11.1 独占、共享与虚拟设备

从资源管理角度，设备可以按使用语义理解：

- **独占设备**：同一时刻通常只能被一个任务占有，如经典打印机模型；
- **共享设备**：可以通过请求队列交错服务多个进程，如磁盘；
- **虚拟设备**：利用 Spooling 等机制让独占物理设备表现出可并发提交的逻辑共享接口。

11.2 设备分配涉及的资源对象

一次 I/O 可能需要的不只是“设备本体”，还可能涉及：

Device + Controller + Channel/Queue Resource

经典设备分配题应明确系统采用静态分配还是动态申请、是否允许排队、是否可能形成资源等待环。

11.3 408 经典设备分配数据结构：把“设备链”画出来

一些教材用以下表结构描述从系统到具体硬件资源的管理关系：

- **SDT (System Device Table)**：系统设备总表，记录设备总体信息及其 DCT 入口；
- **DCT (Device Control Table)**：具体设备状态、等待队列、驱动入口，并关联控制器；
- **COCT (Controller Control Table)**：控制器状态及其连接设备/通道关系；
- **CHCT (Channel Control Table)**：I/O 通道状态和相关资源。

于是经典分配路径可以抽象为：

Process Request → SDT → DCT → COCT → CHCT

实际 OS 实现未必使用这些名字，但考试模型借此强调：**申请一个设备可能隐含申请控制器、通道和队列等多层资源。**

11.4 静态分配与动态分配

- **静态分配**：作业运行前一次性取得所需设备，结束后统一释放；实现简单、可避免某些运行时死锁，但利用率低；
- **动态分配**：运行中按需申请/释放，利用率高，但必须处理等待、分配策略和潜在死锁。

如果多个进程按不同顺序申请设备/控制器/通道，就应立即联想到并发专题中的**资源等待图与死锁条件**。

11.5 为什么块设备需要调度

当多个请求同时存在：

$R_1, R_2, \dots, R_n \longrightarrow \boxed{\text{One/Finite Device Queue}}$

就需要策略决定服务顺序。核心评价可能包括：

- 平均响应/等待时间；
- 吞吐；
- 公平性与饥饿；
- 设备物理成本；
- 优先级/截止时间。

12 第 11 章：HDD 成本模型与经典磁盘调度

12.1 先有物理成本，再有调度策略

经典机械硬盘访问时间：

$T_{access} = T_{seek} + T_{rotation} + T_{transfer}$

其中随机请求常由寻道和旋转等待主导，因此调度器试图减少磁头移动和无效等待。

12.2 六种经典算法的“设计动机”

算法规则	详细决策逻辑	核心设计思想与性能/饥饿风险
FCFS （先来先服务）	按请求到达的绝对先后顺序依次服务	完全公平且实现简单，但磁头可能发生剧烈来回摆动
SSTF （最短寻道优先）	优先选择距离当前磁头位置最近的下一个请求	显著降低局部寻道时间，但两端远端请求可能严重饥饿
SCAN （电梯算法）	磁头沿一个方向移动并服务，到达端点后再反向	解决 SSTF 饥饿问题，改善总体移动效率与响应公平性
C-SCAN （循环扫描）	磁头仅沿单向移动并服务，到达末端后直接快速返回起点	服务等待时间分布比 SCAN 更均匀，避免两端不公平
LOOK （LOOK 调度）	属于 SCAN 的改进，磁头只移动到该方向最后一个请求	避免了 SCAN 无盲目扫到物理端点产生的多余寻道
C-LOOK （C-LOOK 调度）	属于 C-SCAN 的 LOOK 化改进	结合单向扫描与紧凑移动范围，不扫无请求的物理端点

磁盘调度题五步法

1. 在数轴上标出当前磁头和所有请求；
2. 明确当前方向（SCAN/C-SCAN/LOOK 类尤其关键）；
3. 写出服务顺序，而不是边算边猜；
4. 累加相邻位置距离；
5. 最后检查有没有把“物理端点”与“最后一个请求位置”混淆。

12.3 磁盘组织与管理：调度之外还要知道哪些系统动作

经典 OS 还会关注磁盘从“裸介质”变成可用存储的几个步骤：

- **低级/物理格式化**：建立介质扇区等物理记录结构（现代设备大量由厂商完成）；
- **分区**：把逻辑地址空间划分成若干区域；
- **逻辑格式化**：在分区上建立文件系统元数据结构；
- **引导块/启动信息**：为系统启动提供初始加载路径；
- **坏块管理**：检测不可可靠使用的介质区域，并通过重映射、备用块等机制隔离。

这些内容与下一本“文件系统专题”有接口：**I/O/块设备层**提供可读写的逻辑块空间，文件系统再把这些块组织成 **inode**、**目录**和**文件**。

12.4 SSD 之后为什么经典电梯思想变弱

SSD 没有机械寻道和旋转等待，所以：

$$T_{seek} \approx 0, \quad T_{rotation} \approx 0$$

经典“最短磁头移动”不再是主要目标。但现实块层仍可能需要处理：

- 请求合并和队列深度；
- 延迟、公平与优先级；
- 多队列并发；
- 设备内部并行度。

因此 408 的 FCFS/SSTF/SCAN 是建立“策略必须服从设备成本模型”思想的经典载体，不是现代所有 SSD 的唯一调度逻辑。

工程边界：Linux

现代 Linux 块层支持 mq-deadline、BFQ、Kyber、none 等 I/O scheduler；选择取决于设备和目标。工程实现会变化，但问题保持不变：多个 I/O 请求在有限服务能力下如何排序，并权衡吞吐、延迟和公平性。

13 第 12 章：综合题——一次阻塞 read() 的完整路径

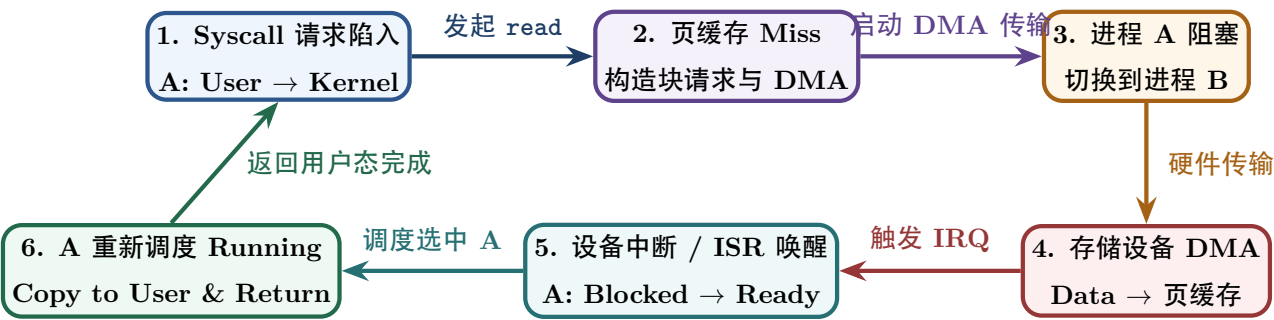
13.1 场景设定

假设进程 A 执行：

```
read(fd, buf, 4096);
```

文件描述符合法，但所需文件页**不在页缓存（页缓存）**，底层设备采用 DMA，调用采用经典阻塞式语义。

13.2 完整流程架构图



13.3 三条时间线合并

时刻	Control / CPU 状态	Data 搬运状态	Process State 进程状态变化
t ₀	A 在用户态执行 read()	尚无数据移动	A: User Running
t ₁	syscall 陷入内核处理	检查文件描述符与 offset	A: Kernel Running (内核上下文)
t ₂	页缓存 Miss, 提交块请求	分配/构造目标缓存页与描述符	A: 尚未能完成返回
t ₃	驱动启动 DMA, 调度器转去 B	设备开始执行 DMA 数据传输	A: Blocked, B: Running
t ₄	CPU 运行进程 B / C 的其他指令	Device $\xrightarrow{\text{DMA}}$ 页缓存	A: Blocked (在等待队列中)
t ₅	设备完成传输, 触发中断进入 ISR	数据已完全入主存 Cache 页	A: Blocked $\rightarrow$ Ready (进入就绪队列)
t ₆	未来 CPU 调度器重新选中进程 A	-	A: Ready $\rightarrow$ Running
t ₇	内核完成路径执行	页缓存 $\xrightarrow{\text{copy}}$ User Buffer	A: Kernel Running
t ₈	系统调用返回用户态	用户进程可直接读 buf 数据	A: User Running

这条路径连接的机制

Syscall + Process State + Scheduler + VFS/File + 页缓存
+ Block Layer + Driver + DMA + Interrupt

能够独立模拟这条路径，说明 I/O 章节不再是“Polling、DMA、Spooling 三个名词”，而已经变成一个真正会运行的系统模型。

14 第 13 章：高频概念辨析与错因诊断

容易混淆的极高频概念对	考场定性标准与正确系统诊断
System Call vs Interrupt	前者通常是程序主动进入内核请求服务；后者是外部/异步硬件事件进入内核。两者都可能触发 Mode Switch，但原因不同。
Interrupt vs Context Switch	中断只保证内核获得处理机会；是否切换进程要看后续调度决策。

容易混淆的极高频概念对	考场定性标准与正确系统诊断
Polling vs PIO	Polling 关注“怎么发现设备完成”；PIO 关注“谁来搬数据”。二者可同时存在。
Interrupt vs Async I/O	设备使用中断完成，不代表用户 API 一定异步；阻塞 <code>read()</code> 也常由中断驱动设备完成。
DMA vs Zero-copy	DMA 只是设备与内存间搬运卸载；用户/内核之间仍可能复制，也可能存在 IOMMU、cache 同步等开销。
Buffer vs Cache	Buffer 主要解决速率/粒度不匹配；Cache 主要保存可能再次被访问的慢对象副本。
Cache vs 页缓存	CPU Cache 缓存物理/地址数据访问的硬件层内容；页缓存是 OS 管理的文件内容缓存。
Spooling vs Buffer	Spooling 不只是“多放一块缓冲区”，而是中转存储 + 队列 + 后台服务，把独占设备虚拟化成可排队服务。
write() return vs durable	buffered write 返回通常只说明内核接受了数据，不代表已落盘到非易失介质。
fsync() vs Writethrough	前者是显式持久性边界接口；后者是缓存写策略术语。
I/O complete vs Process Running	完成事件通常使 waiter Blocked → Ready；真正 Ready → Running 还要调度。
SCAN vs LOOK	SCAN 可走到物理端点；LOOK 只走到该方向最后一个请求。
C-SCAN vs C-LOOK	都单向服务；C-LOOK 同样避免扫到无请求的物理端点。

15 第 14 章：408 I/O 解题检查表

I/O 十三问

拿到任何 I/O 题，建议按以下顺序扫描：

- 1. 主体是谁？哪个进程/线程/设备发起或等待？
- 2. 对象是什么？字符设备、块设备、缓存页、队列、打印任务？
- 3. 请求怎样进入内核？ syscall、fault、已有内核路径？
- 4. 设备由谁控制？ 驱动、控制器、DMA、通道各负责什么？
- 5. 完成怎样被发现？ polling、interrupt、completion queue？
- 6. 数据由谁搬？ CPU/PIO、DMA、内存复制？
- 7. 调用者现在怎么办？ Running、spin、Blocked、立即返回？
- 8. 中间有没有 Buffer/Cache/Queue？ 它解决的是速率、复用还是排队？
- 9. 走快路径还是慢路径？ 页缓存 hit/miss、队列空闲/拥塞？
- 10. 完成后谁的状态改变？ Blocked → Ready 还是直接返回？
- 11. 是否涉及调度策略？ CPU scheduler、disk scheduler、spool queue？
- 12. 正确性边界是什么？ 数据已复制、I/O 已完成、还是已经持久化？
- 13. 成本在哪里？ syscall、中断、复制、DMA、寻道、排队、writeback、context switch？

15.1 题型到核心模型的映射

408 常见题型分类	优先调用的核心核心模型与推导步骤
程序查询 / 中断 / DMA 比较	“等待由谁承担 + 数据由谁搬 + CPU 参与度”三问模型
中断后进程状态变化	Control Flow + Process State Flow，特别检查 Blocked → Ready
单双缓冲计算	先画设备生产、buffer 占用、CPU 消费时间线，再找可重叠区间
SPOOLing 系统原理	Exclusive Device → Queue → Background Service 虚拟化框架
页缓存读写逻辑	File Offset → Cache Hit/Miss；读路径与 dirty/writeback 写路径分开
磁盘调度算法题	HDD cost model → 服务顺序 → 磁头移动距离精确累加
跨计组系统综合	CPU/Bus/DMA/Interrupt + OS Block/Wakeup/Scheduler 共同推演

16 第 15 章：知识树与查漏清单

I/O 知识树

I/O

入口/控制	设备接口	数据传输	性能/解耦	策略/调度
syscall	controller	PIO	buffer	device allocation
interrupt	MMIO/port	DMA	页缓存	disk scheduling
exception	driver	channel	readahead	spool queue
mode switch	char/block	bus contention	writeback	fairness/latency

16.1 408 Core：必须形成自动反应

- I/O 硬件基本组成：设备、控制器、状态/控制/数据接口；
- 程序查询、中断驱动、DMA、I/O 通道的分工和 CPU 参与度；
- 中断完成、阻塞、唤醒与调度之间的状态关系；
- I/O 软件层次：用户 I/O、设备独立层、驱动、中断处理；
- Buffer、双缓冲/循环缓冲的目的；
- 设备分配、独占/共享/虚拟设备；
- SPOOLing 原理及输入/输出井、后台服务思想；
- HDD 访问时间与 FCFS/SSTF/SCAN/C-SCAN/LOOK/C-LOOK；
- 页缓存与 read/write 的接口理解，用于跨章节综合；
- DMA 与计组总线/中断、OS 状态切换的跨科综合。

16.2 Mechanism Layer：理解后做综合题更稳

- Request Path 与 Completion Path；
- Control/Data/State 三条线；
- Polling vs Interrupt 的等待成本模型；
- DMA setup/transfer/completion/cleanup；
- Blocking/Nonblocking 与 Sync/Async 的维度拆分；
- 页缓存 hit/miss、readahead、dirty/writeback；
- write() return、writeback、durability 三个时间点；
- 设备成本模型改变以后调度策略为何必须变化。

16.3 工程扩展：用于限定教材模型的边界

- x86/Linux 具体 syscall/interrupt entry;
- IOMMU 与 DMA address;
- interrupt moderation、polling/interrupt hybrid;
- Linux block multiqueue 与 mq-deadline/BFQ/Kyber/none;
- folio、现代页缓存/writeback 实现;
- O_DIRECT、io_uring、异步完成队列等现代接口。

17 第 16 章：专题接口

17.1 与“进程、线程、调度与控制权”专题

I/O 把抽象的进程状态图变成真实事件：

read waits $\rightarrow$ *Running* $\rightarrow$ *Blocked*

I/O completion $\rightarrow$ *Blocked* $\rightarrow$ *Ready*

scheduler $\rightarrow$ *Ready* $\rightarrow$ *Running*

17.2 与“CPU $\times$ OS 软硬件协作”专题

Hardware: IRQ/DMA/MMIO Kernel: Driver/WaitQueue/Scheduler/Policy

I/O 是软硬件边界最密集的一块。

17.3 与“内存虚拟化”专题

页缓存、DMA buffer、用户 buffer、mmap 都会牵涉：

VA $\rightarrow$ PTE $\rightarrow$ Physical Page

但“文件内容缓存”与“地址翻译缓存”要严格分层。

17.4 与下一本“文件系统”专题

文件系统解决：

Name/FD $\rightarrow$ File Object $\rightarrow$ Logical File Block

本册解决：

I/O Request $\rightarrow$ Transfer $\rightarrow$ Completion

两者在页缓存、块层和 read/write/mmap 路径处汇合。

18 核心结论：I/O 是异步状态转换系统

传统章节容易留下这样的知识碎片：

Polling、Interrupt、DMA、Buffer、Spooling、Disk Scheduling 各背一段。

本册希望建立的是另一种模型：

谁提出请求？	→	谁接管控制？	→	谁搬数据？
	→	谁等待？	→	完成怎样被发现？
			→	状态怎样恢复？

进一步再问：

能否缓存？ 能否批量？ 能否排队？ 能否卸载？
瓶颈在 CPU、内存带宽、设备、队列还是持久性边界？

只要能沿这两组问题完整推演，I/O 就不再是操作系统里最零散的一章，而会变成连接**进程状态、调度、软硬件边界、虚拟内存、文件系统与存储设备**的中心枢纽。

参考与工程边界资料

本册核心结构服务于 408 教学模型；以下资料用于验证现代实现边界，避免把教材抽象误认为唯一工程实现。

1. Linux Kernel Documentation, *Entry/exit handling for exceptions, interrupts, syscalls and KVM*.
<https://docs.kernel.org/core-api/entry.html>
2. Linux Kernel Documentation, *Dynamic DMA mapping Guide / DMA API*.
<https://docs.kernel.org/core-api/dma-api-howto.html>
<https://docs.kernel.org/core-api/dma-api.html>
3. Linux Kernel Documentation, *Buffered I/O / iomap operations*.
<https://docs.kernel.org/filesystems/iomap/operations.html>
4. Linux Kernel Documentation, *Explicit volatile write back cache control*.
https://docs.kernel.org/block/writeback_cache_control.html
5. Linux Kernel Documentation, *Block layer / I/O schedulers*.
<https://docs.kernel.org/block/>
6. Linux Kernel Documentation, *Driver model / driver API*.
<https://docs.kernel.org/driver-api/driver-model/overview.html>
7. IBM Documentation, *Output spooling / output queues*.
<https://www.ibm.com/docs/en/i/7.5?topic=concepts-output-spooling>
8. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, I/O Devices chapters.
<https://pages.cs.wisc.edu/~remzi/OSTEP/>